
NEXUS-1

Object-Oriented Domain Model

Entity Framework Core — Code-First Persistence Design

Subject	A relational domain model for the NEXUS-1 operator-intelligence console, expressed as C# entities under an EF Core Code-First mapping.
Target stack	.NET/C# (EF Core 8), the documented production back-end for the project.
Structure	One unified <code>DbContext</code> (monolith-first), with the natural bounded-context seams flagged for a later microservice split.
Coverage	Every console subsystem — fleet, physics, components, lifecycle, inspection, personnel, robotics, access, monitoring, diagnostics, root cause, compliance, waste and reference data.
Build	NX1-094C79E0A366D34F
Author	Agathangelidis Grigorios

Scope & honesty. This is a *persistence and domain* design — how the simulator's concepts map onto durable, queryable state behind a real .NET back-end. It is demonstration-grade engineering, not a certified plant data model. Where the console's state is genuinely transient (live telemetry, solver internals), the document says so and models the *persisted* projection rather than pretending a row exists for every animation frame.

1 Purpose and modelling stance

The NEXUS-1 console is a single-file in-browser simulator: its state lives in JavaScript objects that are recomputed every tick. This document answers a different question — *if the documented .NET back-end existed, what would the data model look like?* It re-expresses each screen's concepts as a set of C# entity classes mapped to a relational store through Entity Framework Core's Code-First workflow, where the C# types are the source of truth and the schema is generated by migration.

Three principles shape every decision below.

- **Persisted state, not animation state.** The console continuously integrates a point-kinetics solver and redraws gauges. None of that per-tick churn belongs in a relational table. The model persists *configuration* (a unit's design, a component's degradation parameters), *current snapshots* (the latest reconciled state), and *events* (alarms, SCRAMs, audit entries). High-rate signals are explicitly delegated to a time-series path, not row-per-sample tables.
- **Engineered structure is data, inferred structure is not.** The Root Cause graph rests on an auditable fault-tree backbone; learned weights only annotate it. The model mirrors that: causal edges are first-class rows with a provenance discriminator, never silently synthesised.
- **One context now, clean seams for later.** The brief is a single `DbContext`. But the production roadmap targets bounded-context microservices, so each aggregate root is grouped under a logical context and the eventual fault lines are tabulated in §18. Splitting later becomes a mechanical exercise rather than a redesign.

2 Design conventions

These conventions are applied uniformly so the per-domain listings stay terse.

- **Surrogate keys + natural codes.** Every entity carries a `Guid Id` surrogate (distribution-friendly for the later split). Domain identifiers that appear on screen — unit names, FV-104, EVT-2026-0418, DC-07 — are kept as a separate, uniquely-indexed `Code/Tag` string.
- **Enums as strings.** Status, severity, classification and role enums are persisted via `HasConversion<string>()`, so the database stays human-readable and a new member never silently collides with an old integer.
- **Value objects as owned types.** Cohesive bundles with no independent identity — a reactor's live snapshot, a component's degradation coefficients, a hash seal — are EF *owned types*, persisted into their owner's table (or an owned table) rather than promoted to standalone entities.
- **Money-free precision.** Engineering quantities use `decimal` with explicit `HasPrecision(p, s)` where exactness matters (dose budgets, volumes, weights), and `double` where they are model outputs already approximate by nature (flux, reactivity).
- **Auditing & soft delete.** Mutable aggregates implement `IAuditable (CreatedUtc/UpdatedUtc)` and, where history matters, `ISoftDelete`. A `SaveChanges` interceptor stamps these centrally.
- **Optimistic concurrency.** Aggregate roots that operators mutate carry a `[Timestamp] byte[] RowVersion`.
- **Computed properties stay computed.** Derived figures the console calculates live (a component's health %, a sector's coverage verdict) are `[NotMapped]` read-only members; only the inputs and the last committed snapshot are stored.

2.1 Shared base types

```
public abstract class Entity
{
    public Guid Id { get; set; } = Guid.NewGuid();
}

public interface IAuditable
{
    DateTime CreatedUtc { get; set; }
    DateTime? UpdatedUtc { get; set; }
}

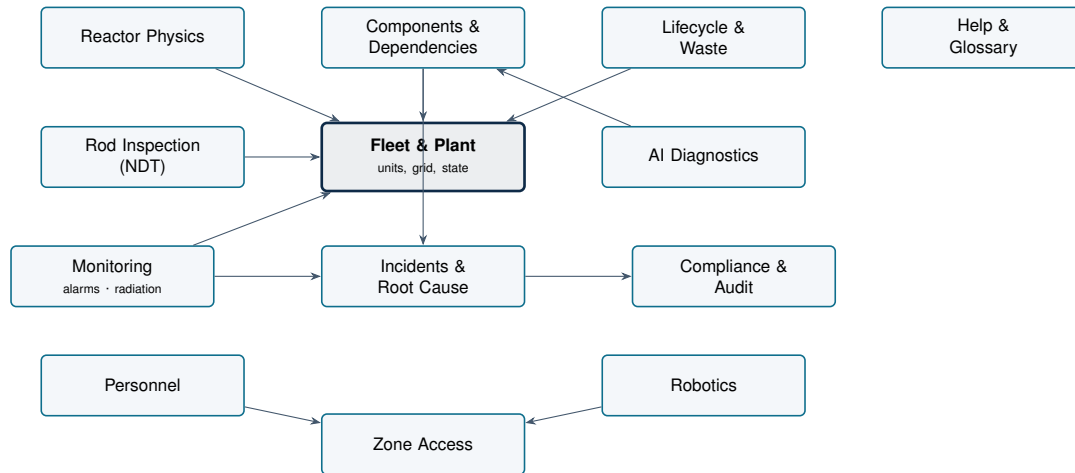
public interface ISoftDelete
{
    bool IsDeleted { get; set; }
    DateTime? DeletedUtc { get; set; }
}

// Carried by aggregate roots operators can mutate concurrently.
public interface IConcurrent
{
}
```

```
byte[] RowVersion { get; set; }
}
```

3 Domain map

The thirteen logical contexts below all live in one `DbContext` for now. Arrows show the strongest cross-context references; the unit selector that drives the console corresponds to the **Fleet** context being referenced almost everywhere.



4 Fleet & Plant

The spine of the model. A `Plant` owns a fleet of reactor `Units` synchronised to one `GridBus`. `Units` come in two shapes — large PWR units and SMR modules — which differ in behaviour and tracked detail, so the hierarchy is mapped **table-per-hierarchy (TPH)** with a discriminator. Each unit owns a `ReactorState` value object: the latest reconciled snapshot, not a telemetry history.

```

public enum UnitClass { Pwr, Smr }
public enum LifecycleStage { Construction, Operating, ExtendedShutdown, Decommissioning, Dismantled }

public class Plant : Entity, IAuditable
{
    public string Name { get; set; } = "";
    public Guid GridBusId { get; set; }
    public GridBus GridBus { get; set; } = null!;
    public ICollection<ReactorUnit> Units { get; set; } = new List<ReactorUnit>();
    public DateTime CreatedUtc { get; set; }
    public DateTime? UpdatedUtc { get; set; }
}

public class GridBus : Entity
{
    public string Name { get; set; } = "400 kV bus";
    public double NominalKv { get; set; } = 400;
    public double FrequencyHz { get; set; } = 50.0; // last committed snapshot
}

// ---- TPH root ----
public abstract class ReactorUnit : Entity, IAuditable, IConcurrent
{
    public string Code { get; set; } = ""; // e.g. "U1", "SMR-3"
    public string Name { get; set; } = "";
    public string ClassLabel { get; set; } = ""; // "PWR 1100 MWe"
    public UnitClass Class { get; set; }
    public double CapacityMwe { get; set; }
    public bool IsOnline { get; set; }
    public LifecycleStage Stage { get; set; } = LifecycleStage.Operating;
    public int ScramCycles { get; set; } // accrued SCRAM thermal cycles
    public double SimOperatingYears { get; set; }

    public Guid PlantId { get; set; }
    public Plant Plant { get; set; } = null!;

    public ReactorState State { get; set; } = new(); // owned snapshot
    public KineticsParameters Kinetics { get; set; } = new(); // owned

    public ICollection<Component> Components { get; set; } = new List<Component>();
    public ICollection<RodAssembly> Rods { get; set; } = new List<RodAssembly>();
    public SpentFuelInventory? SpentFuel { get; set; }
}

```

```

    public DecommissioningProgram? Decommissioning { get; set; }

    public DateTime CreatedUtc { get; set; }
    public DateTime? UpdatedUtc { get; set; }
    public byte[] RowVersion { get; set; } = Array.Empty<byte>();
}

public class PwrUnit : ReactorUnit
{
    public int FuelAssemblies { get; set; } = 157;    // full PWR lattice
    public int CoolantLoops { get; set; } = 3;
}

public class SmrModule : ReactorUnit
{
    public bool IntegralConfiguration { get; set; } = true; // no external loops
    public int ModulesInPack { get; set; } = 1;
}

```

4.1 The live snapshot as an owned value object

`ReactorState` bundles the headline neutronic, thermal-hydraulic, electrical and radiological figures the Overview, Coolant, Power and Radiation screens read. It has no identity of its own — it is always *this unit's* current state — so it is owned, not an entity.

```

public class ReactorState    // EF owned type, stored in the unit's table
{
    // neutronics
    public double FluxPercent { get; set; }
    public double Reactivity { get; set; }           // pcm
    public double RodInsertion { get; set; }         // %
    public double RodTarget { get; set; }
    public double Xenon { get; set; }
    // thermal-hydraulics
    public double CoreTempC { get; set; }
    public double PressureBar { get; set; }
    public double HotLegC { get; set; }
    public double ColdLegC { get; set; }
    public double PressuriserLevel { get; set; }
    public double SteamGenLevel { get; set; }
    public double FeedwaterKgs { get; set; }
    // electrical / grid
    public double ThermalMwt { get; set; }
    public double ElectricalMwe { get; set; }
    public double TurbineRpm { get; set; }
    public double GeneratorMvar { get; set; }
    public double PowerFactor { get; set; }
    // radiological
    public double AreaDoseMsvh { get; set; }
    public DateTime AsOfUtc { get; set; }
}

```

Why no `TelemetrySample` table here. Persisting every gauge tick as a row would be an anti-pattern — it is high-rate, append-only, query-by-window data. In the production stack that stream lands in a time-series store fed off Kafka; the relational model keeps only the *latest reconciled snapshot* (above) plus the digital-twin residuals (§15). The Trends screen reads the time-series path, not this table.

5 Reactor Physics

The console runs two physics codepaths: a rigorous six-group point-kinetics solver and a lighter illustrative model. The solver's *internal* state (precursor concentrations mid-integration) is transient and never persisted. What is worth persisting is the *parameter set* — the constants that define a unit's kinetics — so the model owns a `KineticsParameters` value object with a child collection of delayed-neutron groups.

```

public enum PhysicsModel { PointKinetics, Illustrative }

public class KineticsParameters    // owned by ReactorUnit
{
    public PhysicsModel Model { get; set; } = PhysicsModel.PointKinetics;
    public double GenerationTime { get; set; }           // prompt neutron lifetime, s
    public double DopplerCoeffPcmK { get; set; }         // fuel-temperature feedback
    public double ModeratorCoeffPcmK { get; set; }       // coolant-temperature feedback
    public double XenonWorthPcm { get; set; }
    // six U-235 delayed-neutron precursor groups (owned collection)
    public ICollection<DelayedNeutronGroup> Groups { get; set; }
        = new List<DelayedNeutronGroup>();
}

```

```
public class DelayedNeutronGroup // owned, ordered 1..6
{
    public int Index { get; set; } // 1..6
    public double BetaFraction { get; set; } // beta_i
    public double DecayConstant { get; set; } // lambda_i, 1/s
}
```

Modelled honestly. The two-codepath inconsistency the project documents is preserved, not hidden: `PhysicsModel` records which model a unit is configured for. The recommended `NEXUS.Physics` unification would collapse this to a single member without touching the schema.

6 Components, Dependencies & Aging

Each unit has a registry of tracked `Components`, each mapped to a node in a directed influence graph. The console's aging page tracks a *subset* — the major, mostly passive components (RPV, primary piping, steam generators, containment concrete, cables, core internals) — for long-term life consumed. Rather than duplicate those as a parallel list (the known Registry-vs-Aging overlap), the model gives a component an **optional owned `LifeAssessment`**: present only for the items under long-term surveillance. One inventory, two views.

```
public enum ComponentHealthState { Healthy, Monitor, Degrading, Fault }
public enum DegradationMechanism
{
    None, ThermalFatigue, Embrittlement, StressCorrosionCracking,
    FlowAcceleratedCorrosion, ConcreteAging, CableInsulationAging, Wear
}

public class ComponentSystem : Entity // grouping: primary, secondary, electrical...
{
    public string Name { get; set; } = "";
}

public class Component : Entity, IAuditable
{
    public string Tag { get; set; } = ""; // "FV-104", "RCP-1B"
    public string Name { get; set; } = "";
    public bool IsCritical { get; set; }
    public string? Note { get; set; }

    public Guid UnitId { get; set; }
    public ReactorUnit Unit { get; set; } = null!;
    public Guid SystemId { get; set; }
    public ComponentSystem System { get; set; } = null!;

    // degradation coefficients (owned value object) drive the live health model
    public DegradationModel Degradation { get; set; } = new();
    public double HealthSnapshot { get; set; } = 100; // last committed %
    public ComponentHealthState StateSnapshot { get; set; } = ComponentHealthState.Healthy;

    // present only for the major passive components the Aging screen tracks
    public LifeAssessment? Life { get; set; }

    // graph membership
    public Guid? DependencyNodeId { get; set; }
    public DependencyNode? DependencyNode { get; set; }

    public DateTime CreatedUtc { get; set; }
    public DateTime? UpdatedUtc { get; set; }

    [System.ComponentModel.DataAnnotations.Schema.NotMapped]
    public bool RequiresAttention => StateSnapshot is
        ComponentHealthState.Degrading or ComponentHealthState.Fault;
}

public class DegradationModel // owned: the live-health inputs (w, sc, ld, base, service)
{
    public double WearWeight { get; set; } // % per year base wear
    public double ScramSensitivity { get; set; } // % lost per SCRAM cycle
    public double LoadFactor { get; set; } // flux-coupled wear
    public double AsBuiltOffset { get; set; }
    public double ServiceYears { get; set; }
}

public class LifeAssessment // owned: long-term passive-component surveillance
{
    public DegradationMechanism LimitingMechanism { get; set; }
    public double LifeConsumedPercent { get; set; }
    public int LicensedLifeYears { get; set; }
}
```

6.1 The influence graph

Dependencies are a directed, weighted graph with propagation delays — a self-referencing many-to-many with payload, so it needs an explicit join entity.

```
public class DependencyNode : Entity
{
    public string Key { get; set; } = ""; // "loop-flow", "core-dT"
    public string Label { get; set; } = "";
    public ICollection<InfluenceEdge> Outgoing { get; set; } = new List<InfluenceEdge>();
    public ICollection<InfluenceEdge> Incoming { get; set; } = new List<InfluenceEdge>();
}

public class InfluenceEdge : Entity
{
    public Guid SourceId { get; set; }
    public DependencyNode Source { get; set; } = null!;
    public Guid TargetId { get; set; }
    public DependencyNode Target { get; set; } = null!;
    public double Weight { get; set; } // 0.00 - 1.00
    public double PropagationDelaySec { get; set; }
    public bool IsNegativeFeedback { get; set; }
}
```

7 Rod Inspection (NDT)

Each rod type has a deterministic inventory of rod assemblies; each assembly is examined by the six standard NDT methods, producing a per-method finding and an overall verdict. Findings are a many-to-many between assemblies and methods with payload, so `InspectionResult` is an explicit join entity. Defects detected on an assembly are their own child collection.

```
public enum RodCategory { Control, SafetyShutdown, BurnablePoison, Instrumentation, Source }
public enum NdtMethodKind { Radiography, Ultrasonic, Visual, EddyCurrent, DyePenetrant, MagneticParticle }
public enum InspectionVerdict { Accept, Rework, Reject }

public class RodType : Entity
{
    public RodCategory Category { get; set; }
    public string Material { get; set; } = "";
    public string Cladding { get; set; } = "";
    public ICollection<RodAssembly> Assemblies { get; set; } = new List<RodAssembly>();
}

public class RodAssembly : Entity, IAuditable
{
    public string Serial { get; set; } = "";
    public Guid RodTypeId { get; set; }
    public RodType RodType { get; set; } = null!;
    public Guid? UnitId { get; set; }
    public ReactorUnit? Unit { get; set; }
    public InspectionVerdict Verdict { get; set; } = InspectionVerdict.Accept;
    public int ScramDemandCycles { get; set; } // for control/safety rods
    public ICollection<RodDefect> Defects { get; set; } = new List<RodDefect>();
    public ICollection<InspectionResult> Results { get; set; } = new List<InspectionResult>();
    public DateTime CreatedUtc { get; set; }
    public DateTime? UpdatedUtc { get; set; }
}

public class NdtMethod : Entity
{
    public NdtMethodKind Kind { get; set; }
    public string DetectableDefectClasses { get; set; } = "";
}

public class InspectionResult : Entity // join: assembly x method + finding
{
    public Guid RodAssemblyId { get; set; }
    public RodAssembly RodAssembly { get; set; } = null!;
    public Guid NdtMethodId { get; set; }
    public NdtMethod Method { get; set; } = null!;
    public InspectionVerdict Finding { get; set; }
    public string? Notes { get; set; }
    public DateTime PerformedUtc { get; set; }
}

public class RodDefect : Entity
{
    public Guid RodAssemblyId { get; set; }
    public RodAssembly RodAssembly { get; set; } = null!;
    public string DefectClass { get; set; } = ""; // crack, void, inclusion...
    public double? SizeMm { get; set; }
}
```

8 Personnel

Staffing is organised by sector. Each sector declares role *requirements* (how many of a role, and whether it is safety-critical) and holds a roster of named staff. The coverage verdict (MET / DEGRADED / BREACH) the stress test produces is computed, not stored.

```
public enum ShiftPattern { Day, RotatingShift, OnCall }

public class Sector : Entity
{
    public string Name { get; set; } = ""; // Operations, Maintenance, RP...
    public int MinimumComplement { get; set; }
    public ICollection<RoleRequirement> Requirements { get; set; } = new List<RoleRequirement>();
    public ICollection<StaffMember> Staff { get; set; } = new List<StaffMember>();
}

public class RoleRequirement : Entity
{
    public Guid SectorId { get; set; }
    public Sector Sector { get; set; } = null!;
    public string Role { get; set; } = ""; // "Reactor Operator (RO)"
    public int RequiredCount { get; set; }
    public bool IsCritical { get; set; }
}

public class StaffMember : Entity, IAuditable, ISoftDelete
{
    public Guid SectorId { get; set; }
    public Sector Sector { get; set; } = null!;
    public string Name { get; set; } = "";
    public string Role { get; set; } = "";
    public string LicenceCategory { get; set; } = ""; // SRO, RO, STA, I&C...
    public ShiftPattern Shift { get; set; }
    public bool IsPresent { get; set; } = true;
    public string? AbsenceReason { get; set; }
    public DateTime CreatedUtc { get; set; }
    public DateTime? UpdatedUtc { get; set; }
    public bool IsDeleted { get; set; }
    public DateTime? DeletedUtc { get; set; }
}
```

9 Robotics & Vehicles

The robotic fleet is grouped by category. Each asset has a battery charge and a *cumulative* radiation dose against a finite tolerance. Missions declare the capabilities they require; assets advertise capabilities. Both relations are many-to-many over a shared Capability vocabulary — modelled with skip navigations (EF Core synthesises the join tables).

```
public enum RobotStatus { Available, Deployed, Charging, Maintenance, Retired }

public class RobotCategory : Entity
{
    public string Name { get; set; } = ""; // Inspection, Survey, Decon, Emergency...
    public ICollection<RobotAsset> Assets { get; set; } = new List<RobotAsset>();
}

public class Capability : Entity
{
    public string Code { get; set; } = ""; // "high-dose", "manipulation", "cutting"
    public ICollection<RobotAsset> Assets { get; set; } = new List<RobotAsset>();
    public ICollection<Mission> Missions { get; set; } = new List<Mission>();
}

public class RobotAsset : Entity, IAuditable, IConcurrent
{
    public string Tag { get; set; } = ""; // "DC-07"
    public string Name { get; set; } = "";
    public string Type { get; set; } = "";
    public Guid CategoryId { get; set; }
    public RobotCategory Category { get; set; } = null!;
    public string Location { get; set; } = "";
    public RobotStatus Status { get; set; }
    public double BatteryPercent { get; set; }
    public double CumulativeDoseSv { get; set; }
    public double DoseToleranceSv { get; set; }
    public ICollection<Capability> Capabilities { get; set; } = new List<Capability>();
    public DateTime CreatedUtc { get; set; }
    public DateTime? UpdatedUtc { get; set; }
    public byte[] RowVersion { get; set; } = Array.Empty<byte>();

    [System.ComponentModel.DataAnnotations.Schema.NotMapped]
    public double RemainingDoseBudgetSv => DoseToleranceSv - CumulativeDoseSv;
}
```

```
public class Mission : Entity
{
    public string Name { get; set; } = "";
    public bool IsCritical { get; set; }
    public double MinBatteryPercent { get; set; }
    public double DoseCostSv { get; set; }
    public ICollection<Capability> RequiredCapabilities { get; set; } = new List<Capability>();
}
```

10 Zone Access

A permissions matrix: which class of entity (person role, vehicle, robot) may enter which controlled zone. That matrix is a many-to-many with an `IsAllowed` payload, so `ZonePermission` is explicit. Live occupancy is a stream of `PresenceRecords`; one flagged against policy becomes an `AccessViolation`.

```
public enum ZoneRestriction { Public, Controlled, Containment, VitalArea }
public enum EntityKind { Personnel, Vehicle, Robot, Contractor }

public class Zone : Entity
{
    public string Name { get; set; } = "";
    public ZoneRestriction Restriction { get; set; }
    public ICollection<ZonePermission> Permissions { get; set; } = new List<ZonePermission>();
}

public class AccessEntityClass : Entity
{
    public string Name { get; set; } = ""; // "Reactor Operator", "Logistics Vehicle"
    public EntityKind Kind { get; set; }
    public ICollection<ZonePermission> Permissions { get; set; } = new List<ZonePermission>();
}

public class ZonePermission : Entity // join: zone x entity-class + allowed
{
    public Guid ZoneId { get; set; }
    public Zone Zone { get; set; } = null!;
    public Guid EntityClassId { get; set; }
    public AccessEntityClass EntityClass { get; set; } = null!;
    public bool IsAllowed { get; set; }
}

public class PresenceRecord : Entity
{
    public Guid ZoneId { get; set; }
    public Zone Zone { get; set; } = null!;
    public Guid EntityClassId { get; set; }
    public AccessEntityClass EntityClass { get; set; } = null!;
    public string EntityLabel { get; set; } = ""; // badge / asset id
    public DateTime EnteredUtc { get; set; }
    public DateTime? ExitedUtc { get; set; }
    public bool IsViolation { get; set; }
}
```

11 Monitoring — Alarms & Radiation

Alarms are events with a severity and an acknowledgement lifecycle, optionally bound to a source component or dependency node (the binding is what the Root Cause graph reasons over). Radiation areas hold dose readings against limits.

```
public enum AlarmSeverity { Info, Low, Medium, High, Critical }
public enum AlarmState { Active, Acknowledged, Cleared }

public class Alarm : Entity, IAuditable
{
    public string Code { get; set; } = ""; // "ALM-3391"
    public string Message { get; set; } = "";
    public AlarmSeverity Severity { get; set; }
    public AlarmState State { get; set; } = AlarmState.Active;
    public DateTime RaisedUtc { get; set; }
    public DateTime? AcknowledgedUtc { get; set; }
    public string? AcknowledgedBy { get; set; }
    public Guid? UnitId { get; set; }
    public ReactorUnit? Unit { get; set; }
    public Guid? SourceComponentId { get; set; }
    public Component? SourceComponent { get; set; }
    public DateTime CreatedUtc { get; set; }
    public DateTime? UpdatedUtc { get; set; }
}
```



```

public class RadiationArea : Entity
{
    public string Name { get; set; } = "";
    public double DoseLimitMsvh { get; set; }
    public ICollection<DoseReading> Readings { get; set; } = new List<DoseReading>();
}

public class DoseReading : Entity
{
    public Guid AreaId { get; set; }
    public RadiationArea Area { get; set; } = null!;
    public double DoseRateMsvh { get; set; }
    public DateTime MeasuredUtc { get; set; }
}

```

12 Incidents & Root Cause

An Incident (e.g. EVT-2026-0418) has an ordered timeline of IncidentEvents — live SCRAMs append here automatically. Its causal reconstruction is a graph: CausalNodes connected by CausalEdges whose **provenance** is a first-class discriminator (engineered backbone, data-informed, artefact, or rejected candidate), exactly as the methodology insists. Ranked RootCauseHypothesis rows hold the triage figures (direct alarms, alarms explained, illustrative confidence).

```

public enum IncidentEventType { Scram, AlarmBurst, OperatorAction, SystemAction, Annotation }
public enum CausalEdgeProvenance { FaultTreeBackbone, DataInformed, Artefact, RejectedCandidate }
public enum HypothesisRole { Origin, Proximate, Parallel, Contributing, RuledOut }

public class Incident : Entity, IAuditable
{
    public string Code { get; set; } = ""; // "EVT-2026-0418"
    public string Title { get; set; } = "";
    public string ReasoningClass { get; set; } = ""; // cascade / EMI artefact / QA escape
    public Guid? UnitId { get; set; }
    public ReactorUnit? Unit { get; set; }
    public DateTime OpenedUtc { get; set; }
    public DateTime? ClosedUtc { get; set; }
    public ICollection<IncidentEvent> Timeline { get; set; } = new List<IncidentEvent>();
    public ICollection<CausalNode> Nodes { get; set; } = new List<CausalNode>();
    public ICollection<RootCauseHypothesis> Hypotheses { get; set; } = new List<RootCauseHypothesis>();
    public DateTime CreatedUtc { get; set; }
    public DateTime? UpdatedUtc { get; set; }
}

public class IncidentEvent : Entity
{
    public Guid IncidentId { get; set; }
    public Incident Incident { get; set; } = null!;
    public int Sequence { get; set; }
    public IncidentEventType Type { get; set; }
    public string Description { get; set; } = "";
    public DateTime OccurredUtc { get; set; }
}

public class CausalNode : Entity
{
    public Guid IncidentId { get; set; }
    public Incident Incident { get; set; } = null!;
    public string Label { get; set; } = ""; // "Valve FV-104 (actuator latency)"
    public Guid? ComponentId { get; set; }
    public Component? Component { get; set; } // provenance back to inventory
    public ICollection<CausalEdge> Outgoing { get; set; } = new List<CausalEdge>();
    public ICollection<CausalEdge> Incoming { get; set; } = new List<CausalEdge>();
}

public class CausalEdge : Entity
{
    public Guid FromNodeId { get; set; }
    public CausalNode FromNode { get; set; } = null!;
    public Guid ToNodeId { get; set; }
    public CausalNode ToNode { get; set; } = null!;
    public CausalEdgeProvenance Provenance { get; set; }
    public double Weight { get; set; }
    public double? PropagationDelaySec { get; set; }
}

public class RootCauseHypothesis : Entity
{
    public Guid IncidentId { get; set; }
    public Incident Incident { get; set; } = null!;
    public Guid CandidateNodeId { get; set; }
    public CausalNode CandidateNode { get; set; } = null!;
    public HypothesisRole Role { get; set; }
    public int DirectAlarms { get; set; }
}

```

```

public int AlarmsExplained { get; set; }
public int AlarmsTotal { get; set; }
public double IllustrativeConfidence { get; set; } // 0..1, NOT calibrated
}

```

Confidence is illustrative. *IllustrativeConfidence* is a ranking weight derived from edge strengths — deliberately *not* a probability. The column name carries that caveat into the schema so no downstream query mistakes it for a PRA result.

13 Compliance & Audit

A tamper-evident, append-only log. Each entry seals itself with a hash that references the previous entry's hash, forming a chain — any alteration breaks it. The hash and its predecessor are an owned *HashSeal*. Entries are write-once: no *IAuditable*/*ISoftDelete*, and updates are blocked at the context level.

```

public enum AuditEntryType { Operator, System, Event, Safety, Regulatory }

public class AuditLogEntry : Entity
{
    public long Sequence { get; set; } // monotonic, unique
    public DateTime TimestampUtc { get; set; }
    public string Actor { get; set; } = ""; // "Operator 4417", "System"
    public string Action { get; set; } = "";
    public AuditEntryType Type { get; set; }
    public HashSeal Seal { get; set; } = new(); // owned
}

public class HashSeal // owned value object
{
    public string Hash { get; set; } = "";
    public string PreviousHash { get; set; } = "";
}

```

14 Lifecycle — Waste & Decommissioning

Each unit has at most one *SpentFuelInventory* (pool occupancy, dry-cask count, heavy-metal tonnage) and, once retired, one *DecommissioningProgram* walking ordered *DecommissioningStages*. Fleet waste is held by classification. Decay curves the Waste screen plots are computed from discharge dates, not stored point-by-point.

```

public enum WasteClass { VeryLowLevel, LowLevel, IntermediateLevel, HighLevel }
public enum DecommissioningPhase { Planning, Defuelling, Decontamination, Dismantling, SiteRelease }

public class SpentFuelInventory : Entity
{
    public Guid UnitId { get; set; }
    public ReactorUnit Unit { get; set; } = null!;
    public int PoolAssemblies { get; set; }
    public int PoolCapacity { get; set; }
    public int DryCasks { get; set; }
    public double HeavyMetalTonnes { get; set; } // tHM
    public double CumulativeBurnup { get; set; } // drives the inventory
    public DateTime? LastDischargeUtc { get; set; }
}

public class WasteRecord : Entity
{
    public Guid? UnitId { get; set; }
    public ReactorUnit? Unit { get; set; }
    public WasteClass Classification { get; set; }
    public double VolumeM3 { get; set; }
    public DateTime AsOfUtc { get; set; }
}

public class DecommissioningProgram : Entity, IAuditable
{
    public Guid UnitId { get; set; }
    public ReactorUnit Unit { get; set; } = null!;
    public DecommissioningPhase CurrentPhase { get; set; }
    public DateTime StartedUtc { get; set; }
    public ICollection<DecommissioningStage> Stages { get; set; } = new List<DecommissioningStage>();
    public DateTime CreatedUtc { get; set; }
    public DateTime? UpdatedUtc { get; set; }
}

public class DecommissioningStage : Entity
{
    public Guid ProgramId { get; set; }
    public DecommissioningProgram Program { get; set; } = null!;
}

```

```

public DecommissioningPhase Phase { get; set; }
public int Order { get; set; }
public bool IsComplete { get; set; }
public double RadiologicalActivityFraction { get; set; } // decays over time
}

```

15 AI Diagnostics

The working predictive layer projects a failure probability and anomaly score per component; the digital twin commits model-vs-telemetry residuals. Both are *snapshots* the screens read, refreshed by a background job — not the raw stream.

```

public class DiagnosticForecast : Entity, IAuditable
{
    public Guid ComponentId { get; set; }
    public Component Component { get; set; } = null!;
    public double FailureProbability30d { get; set; }
    public double AnomalyScore { get; set; }
    public DateTime ComputedUtc { get; set; }
    public DateTime CreatedUtc { get; set; }
    public DateTime? UpdatedUtc { get; set; }
}

public class TwinResidual : Entity
{
    public Guid UnitId { get; set; }
    public ReactorUnit Unit { get; set; } = null!;
    public string Signal { get; set; } = ""; // "core-dT", "loop-flow"
    public double Measured { get; set; }
    public double Modelled { get; set; }
    public double Tolerance { get; set; }
    public DateTime AsOfUtc { get; set; }

    [System.ComponentModel.DataAnnotations.Schema.NotMapped]
    public double Residual => Measured - Modelled;
    [System.ComponentModel.DataAnnotations.Schema.NotMapped]
    public bool OutOfTolerance => Math.Abs(Residual) > Tolerance;
}

```

16 Reference — OT Security, Help & Glossary

Mostly static reference data. OT-security controls and their posture; the shared help model keyed by page id; the glossary of ~100 terms across ~15 sections.

```

public enum ControlStatus { Active, Degraded, Planned }

public class SecurityControl : Entity
{
    public string Name { get; set; } = ""; // Zero-trust, Segmentation, HSM, MFA, IDS
    public ControlStatus Status { get; set; }
    public string? Detail { get; set; }
}

public class HelpEntry : Entity
{
    public string PageId { get; set; } = ""; // shared HELP key: "core", "rcgraph"
    public string Title { get; set; } = "";
    public string Summary { get; set; } = "";
    public List<string> KeyPoints { get; set; } = new(); // mapped as JSON column
}

public class GlossarySection : Entity
{
    public string Name { get; set; } = "";
    public int Order { get; set; }
    public ICollection<GlossaryTerm> Terms { get; set; } = new List<GlossaryTerm>();
}

public class GlossaryTerm : Entity
{
    public Guid SectionId { get; set; }
    public GlossarySection Section { get; set; } = null!;
    public string Term { get; set; } = "";
    public string Definition { get; set; } = "";
}

```

17 The unified DbContext

One context exposes every aggregate. `OnModelCreating` applies the conventions that annotations alone cannot express: the TPH discriminator, owned types and owned collections, the self-referencing graphs (with `Restrict delete` to avoid multiple cascade paths), the append-only audit chain, enum-to-string conversions and decimal precision. Configuration is split into `IEntityTypeConfiguration<T>` classes in production; key fragments are shown inline here for brevity.

```
public class NexusDbContext : DbContext
{
    public NexusDbContext(DbContextOptions<NexusDbContext> options) : base(options) { }

    // Fleet & physics
    public DbSet<Plant> Plants => Set<Plant>();
    public DbSet<GridBus> GridBuses => Set<GridBus>();
    public DbSet<ReactorUnit> Units => Set<ReactorUnit>(); // TPH root
    // Components, dependencies, aging
    public DbSet<Component> Components => Set<Component>();
    public DbSet<ComponentSystem> ComponentSystems => Set<ComponentSystem>();
    public DbSet<DependencyNode> DependencyNodes => Set<DependencyNode>();
    public DbSet<InfluenceEdge> InfluenceEdges => Set<InfluenceEdge>();
    // Inspection
    public DbSet<RodType> RodTypes => Set<RodType>();
    public DbSet<RodAssembly> RodAssemblies => Set<RodAssembly>();
    public DbSet<NdtMethod> NdtMethods => Set<NdtMethod>();
    public DbSet<InspectionResult> InspectionResults => Set<InspectionResult>();
    // People, robots, access
    public DbSet<Sector> Sectors => Set<Sector>();
    public DbSet<StaffMember> Staff => Set<StaffMember>();
    public DbSet<RobotCategory> RobotCategories => Set<RobotCategory>();
    public DbSet<RobotAsset> RobotAssets => Set<RobotAsset>();
    public DbSet<Capability> Capabilities => Set<Capability>();
    public DbSet<Mission> Missions => Set<Mission>();
    public DbSet<Zone> Zones => Set<Zone>();
    public DbSet<AccessEntityClass> AccessEntityClasses => Set<AccessEntityClass>();
    public DbSet<ZonePermission> ZonePermissions => Set<ZonePermission>();
    public DbSet<PresenceRecord> PresenceRecords => Set<PresenceRecord>();
    // Monitoring, incidents, audit
    public DbSet<Alarm> Alarms => Set<Alarm>();
    public DbSet<RadiationArea> RadiationAreas => Set<RadiationArea>();
    public DbSet<Incident> Incidents => Set<Incident>();
    public DbSet<CausalNode> CausalNodes => Set<CausalNode>();
    public DbSet<CausalEdge> CausalEdges => Set<CausalEdge>();
    public DbSet<RootCauseHypothesis> Hypotheses => Set<RootCauseHypothesis>();
    public DbSet<AuditLogEntry> AuditLog => Set<AuditLogEntry>();
    // Lifecycle, diagnostics, reference
    public DbSet<SpentFuelInventory> SpentFuel => Set<SpentFuelInventory>();
    public DbSet<WasteRecord> WasteRecords => Set<WasteRecord>();
    public DbSet<DecommissioningProgram> Decommissioning => Set<DecommissioningProgram>();
    public DbSet<DiagnosticForecast> Forecasts => Set<DiagnosticForecast>();
    public DbSet<TwinResidual> TwinResiduals => Set<TwinResidual>();
    public DbSet<SecurityControl> SecurityControls => Set<SecurityControl>();
    public DbSet<HelpEntry> HelpEntries => Set<HelpEntry>();
    public DbSet<GlossaryTerm> GlossaryTerms => Set<GlossaryTerm>();

    protected override void OnModelCreating(ModelBuilder b)
    {
        // ---- TPH: PWR vs SMR share one table ----
        b.Entity<ReactorUnit>()
            .HasDiscriminator<string>("UnitType")
            .HasValue<PwrUnit>("PWR")
            .HasValue<SmrModule>("SMR");
        b.Entity<ReactorUnit>().HasIndex(u => u.Code).IsUnique();
        b.Entity<ReactorUnit>().OwnsOne(u => u.State); // snapshot
        b.Entity<ReactorUnit>().OwnsOne(u => u.Kinetics, // owned collection
            k => k.OwnsMany(x => x.Groups));

        // ---- Component: owned coefficients + optional life assessment ----
        b.Entity<Component>().OwnsOne(c => c.Degradation);
        b.Entity<Component>().OwnsOne(c => c.Life); // null => not tracked
        b.Entity<Component>().HasIndex(c => c.Tag);
        b.Entity<Component>().Property(c => c.StateSnapshot).HasConversion<string>();

        // ---- self-referencing influence graph: no cascade ----
        b.Entity<InfluenceEdge>()
            .HasOne(e => e.Source).WithMany(n => n.Outgoing)
            .HasForeignKey(e => e.SourceId).OnDelete(DeleteBehavior.Restrict);
        b.Entity<InfluenceEdge>()
            .HasOne(e => e.Target).WithMany(n => n.Incoming)
            .HasForeignKey(e => e.TargetId).OnDelete(DeleteBehavior.Restrict);

        // ---- causal graph mirrors the same pattern ----
        b.Entity<CausalEdge>()
            .HasOne(e => e.FromNode).WithMany(n => n.Outgoing)
            .HasForeignKey(e => e.FromNodeId).OnDelete(DeleteBehavior.Restrict);
    }
}
```

```

        b.Entity<CausalEdge>()
            .HasOne(e => e.ToNode).WithMany(n => n.Incoming)
            .HasForeignKey(e => e.ToNodeId).OnDelete(DeleteBehavior.Restrict);

        // ---- append-only, hash-chained audit log ----
        b.Entity<AuditLogEntry>().OwnsOne(a => a.Seal);
        b.Entity<AuditLogEntry>().HasIndex(a => a.Sequence).IsUnique();

        // ---- help key points stored as a JSON column ----
        b.Entity<HelpEntry>().Property(h => h.KeyPoints).HasColumnType("jsonb");

        // ---- blanket conventions: enums as strings ----
        foreach (var et in b.Model.GetEntityTypes())
            foreach (var p in et.GetProperties())
                if (p.ClrType.IsEnum)
                    p.SetProviderClrType(typeof(string));

        // ---- global soft-delete filter ----
        b.Entity<StaffMember>().HasQueryFilter(s => !s.IsDeleted);
    }
}

```

17.1 Write-once enforcement and central auditing

A `SaveChanges` override stamps `IAuditable` timestamps, refuses to mutate or delete `AuditLogEntry`, and computes the next hash seal — keeping that invariant out of every call site.

```

public override int SaveChanges()
{
    foreach (var e in ChangeTracker.Entries<AuditLogEntry>())
        if (e.State is EntityState.Modified or EntityState.Deleted)
            throw new InvalidOperationException("Audit log is append-only.");

    var now = DateTime.UtcNow;
    foreach (var e in ChangeTracker.Entries<IAuditable>())
    {
        if (e.State == EntityState.Added) e.Entity.CreatedUtc = now;
        if (e.State == EntityState.Modified) e.Entity.UpdatedUtc = now;
    }
    return base.SaveChanges();
}

```

18 Microservice seams for the later split

The single context decomposes cleanly along aggregate boundaries. Each row below becomes its own service and database in the production roadmap; the cross-context references collapse to message contracts (a unit id on a Kafka event) rather than foreign keys.

Bounded context	Aggregate roots	Cross-context links
Fleet & Plant	Plant, ReactorUnit, GridBus	referenced by nearly all
Reactor Physics	(owned by ReactorUnit)	—
Asset & Reliability	Component, DependencyNode, ComponentSystem	UnitId
Inspection	RodType, RodAssembly, NdtMethod	UnitId
Personnel	Sector, StaffMember	EntityClass → Access
Robotics	RobotAsset, Mission, Capability	EntityClass → Access
Zone Access	Zone, AccessEntityClass, PresenceRecord	—
Monitoring	Alarm, RadiationArea	UnitId, ComponentId
Incident & Root-cause	Incident, CausalNode, Hypothesis	ComponentId, AlarmId
Compliance & Audit	AuditLogEntry	receives events from all
Lifecycle & Waste	SpentFuelInventory, DecommissioningProgram, WasteRecord	UnitId
Diagnostics & Twin	DiagnosticForecast, TwinResidual	ComponentId, UnitId
Reference	SecurityControl, HelpEntry, GlossaryTerm	—

What this design deliberately avoids. No row-per-tick telemetry tables (time-series path instead); no learned causal edges promoted into the backbone (provenance discriminator guards it); no duplicated component list for ageing (one inventory, optional owned `LifeAssessment`); and no mutable audit log (write-once at the context). Each is a known trap this model sidesteps on purpose.